

AGENT DESIGN LAB

Building the Four Agent Architectures for Your Company

THE TASK

You're working in a team of three, in one shared GitHub repo, on the company and industry you were given. Your job is to invent a single realistic problem that company would actually face and then solve that same problem four separate times once with each agent architecture from the previous session. You're not trying to find the "best" one. You're trying to feel, in your own code, why each architecture behaves differently when it hits the exact same problem.

Model choice is entirely up to you. Use Claude, GPT, Gemini, an open-weight model through Groq or OpenRouter, or something running locally on your own machine. pick whatever you're comfortable with, or use the chance to try something new.

THE FOUR ARCHITECTURES

- **1. Reactive (Rule-Based) Agent:** a pure if/then decision loop, no model call at all. Hard-coded conditions map directly to actions.
- **2. Unconstrained LLM-Powered Agent:** a free-form ReAct-style loop where the model chooses its own reasoning, tool calls, and stopping point, with no schema, no allow-list, and no step limit.
- **3. Deterministic Routing Agent:** a single constrained model call classifies the situation into one of a fixed set of categories; everything after that is ordinary, testable code.
- **4. Constrained ReAct Agent:** the reasoning loop stays, but every step is schema-validated, tool calls are restricted to an allow-list, a MAX_STEPS budget is enforced, and the agent must end in an explicit final answer or an escalation.

A WORKED EXAMPLE — READ THIS, THEN DO SOMETHING DIFFERENT

This example is here to show you the shape of the exercise, not to give you the problem to copy. Pick something genuinely different for your own company — a different kind of decision, not a message-routing problem like the one below.

Picture "PedalPoint," a city bike-share network (not one of the companies on your list). Every bike reports a fault code when something goes wrong — low battery, a jammed lock, GPS drift, a cracked frame sensor. Someone has to decide what happens next: reset it remotely, send a technician, or pull it from the fleet until it's inspected. That decision depends on the fault code, how often that bike has failed recently, and where it currently is.

- **Reactive version:** fault code LOW_BATTERY → send reset command; JAMMED_LOCK or GPS_DRIFT → flag for technician; anything else → pull from fleet. Falls apart the moment two fault codes fire together, or a bike has failed five times this week but reports a code the rules treat as minor.
- **Unconstrained LLM version:** the model reads the fault report plus recent history, freely decides which tool to call (reset device, check failure history, dispatch technician, retire bike), and freely decides when it's satisfied. Handles the two-faults-at-once case well, the LLM could also decide the technician route based on proximity and problem similarity.
- **Deterministic routing version:** one classification call sorts the report into RESET, DISPATCH, or RETIRE, and each label runs a fixed, testable function. Fast and predictable, but it can't factor in the bike's failure history the way the constrained ReAct version can.
- **Constrained ReAct version:** same reasoning loop as the unconstrained version; check history, weigh the fault code, decide. But every step must match a defined schema, tools are limited to an allow-list of three functions, MAX_STEPS = 6, and the loop must end in final_answer or escalate.

WHAT TO BUILD AND SUBMIT

- A short README describing your company, the problem your team invented, and why it genuinely needs an agent rather than a simple script.
- Four folders: reactive/, unconstrained_react/, routing/, constrained_react/ each runnable on its own, with a short note on how to run it and which model/provider it expects.
- In the constrained ReAct folder specifically: the validation schema, the tool allow-list, and the MAX_STEPS constant should be easy to find in the code, not buried.
- A short comparison table in the main README: calls made per request, rough cost or token usage, latency, and what broke when you fed it a tricky or unseen input.
- Commit history should show real contributions from all three of you split the four builds plus the presentation. so no one owns more than two. No one is responsible for a "presentation" only, and every body should be aware **extensively** of what their team created and able to answer whatever comes their way.

THE PRESENTATION

Each team also presents, and the presentation is a story, not a slide dump of code. Put yourselves inside the company: you three work there, this problem landed on your desk, and you're now telling your colleagues how it went. Present it in first person and in order, roughly like this:

- **The setup:** who you are at the company and what day-to-day looked like before anyone thought to use an agent. Make the problem feel real: what was going wrong, who was affected, why it mattered enough to fix.
- **The first attempt:** you tried the simple, rule-based version first, because it was cheap and fast to build. Show us where it broke, in your own test case, not just in theory.
- **Reaching for the model:** you gave the model free rein next. Tell us what got better and be honest about the new problems that showed up such as cost, latency, or a run that went somewhere you didn't expect.
- **Finding the right amount of control:** how you landed on routing, or constraint, or both, and why that felt like the right amount of freedom to give the model for your specific problem.
- **Where it stands now:** what you'd still worry about if this ran in production tomorrow, and what you'd want to try next.

Bring your comparison table as evidence inside the story, not as a separate appendix. Also a live demo or short recording (preferably so you don't hit rate limits or internet problems while presenting) of the agent hitting a tricky input land better than a slide describing what it does.

FREE WAYS TO RUN A MODEL

- **Google AI Studio / Gemini API:** generous free tier, good for both the routing call and the ReAct loop.
- **Mistral — La Plateforme** : a free "Experiment" tier with rate-limited access to their models (Small, Medium, and Large), no credit card required to start; good if you want a European, open-weight-friendly option.
- **Groq** free tier, very fast inference on open-weight models like Llama and Mixtral; handy for cutting latency in the ReAct versions.
- **OpenRouter:** one API that gives you access to several providers, including a number of free-tier open models, useful if you want to compare more than one model on the same agent.
- **Ollama:** run open-weight models locally, fully free and offline, no rate limits to worry about while you're debugging.
- **LM Studio:** a GUI alternative to Ollama for running local models, if you'd rather not live in the terminal for that part.
- **Hugging Face Inference API:** free-tier hosted inference for a wide range of open models.

TUTORIALS, IF YOU'D RATHER WATCH OR LISTEN

Reading docs isn't the only way in. If you learn better by watching someone build the thing, these searches will get you there faster than any single link we could hand you (this space moves fast, so a live search beats a fixed video).

- **YouTube "building an LLM agent with tool use"** : walkthroughs of the reasoning/acting/observing loop in code.
- **YouTube "structured output JSON mode LLM tutorial"** : useful for the schema-validation piece of your constrained ReAct agent.
- **YouTube "run local LLM Ollama tutorial"** : if you want to try the local-model route hands-on.

A FEW MORE RESOURCES

- **Pydantic** — the fastest way to define and validate the step schema your constrained ReAct agent needs.
- **ReAct paper (arXiv 2210.03629)** — the original reasoning/acting/observing loop behind both of your ReAct agents.
- **tenacity (PyPI)** — a small Python library for the bounded-retry logic your constrained agent needs when a step fails validation.
- **python-dotenv** — keep whichever API key you use out of the repo entirely.

A FEW GUARDRAILS

- Pick a problem where the next step genuinely depends on a previous result, otherwise your "constrained ReAct" agent is really just routing wearing a costume.
- Never commit an API key, to any provider. Use a .env file and add it to .gitignore.
- Keep the same set of test inputs across all four agents so your comparison table is a fair, apples-to-apples comparison.